

C++ Programming

Day 3: 배열과 포인터

배열, 포인터 연산, 함수로의 배열 전달, const 포인터

Contents

1	학습 목표	2
2	포인터의 기초	2
2.1	변수와 메모리 주소	2
2.2	포인터 선언	2
2.3	역참조	2
2.4	최종 예제: 01_pointer_basic.cpp	3
3	배열의 기초	4
3.1	배열의 필요성	4
3.2	인덱스	4
3.3	배열 원소 수정	4
3.4	반복문을 이용한 배열 순회	4
3.5	최종 예제: 02_array_basic.cpp	4
4	포인터 연산과 배열	5
4.1	자료형의 크기: sizeof	5
4.2	배열의 연속 저장	6
4.3	배열 이름과 첫 번째 원소의 주소	6
4.4	포인터의 이동	6
4.5	인덱싱과 포인터 표현	6
4.6	포인터 위치를 다시 초기화하기	7
4.7	최종 예제: 03_pointer_arithmetic.cpp	7
5	배열을 함수에 전달하기	8
5.1	배열 출력 함수	8
5.2	함수 매개변수의 배열 표기	9
5.3	배열 크기를 함께 전달하는 이유	9
5.4	함수에서 원본 배열 수정	9
5.5	최종 예제: 04_function_array.cpp	9
6	const 포인터	10
6.1	왜 const를 사용하는가	10
6.2	가리키는 값을 변경할 수 없는 포인터	11
6.3	포인터 자체를 변경할 수 없는 포인터	11
6.4	값과 포인터 모두 변경할 수 없는 경우	11
6.5	세 선언 비교	11
6.6	최종 예제: 05_const_pointer.cpp	11
7	프로젝트 파일 구성	12
8	핵심 정리	13

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 변수의 주소와 포인터의 기본 구조
- 주소 연산자 &와 역참조 연산자 *
- 배열의 선언, 인덱싱, 수정
- 배열 이름과 첫 번째 원소 주소의 관계
- 포인터 연산과 자료형 크기의 관계
- arr[i]와 *(arr + i)의 동치
- 배열을 함수에 전달하는 방식
- const int*, int* const, const int* const의 차이

포인터는 단순히 주소를 저장하는 변수에 그치지 않는다. 배열을 순회하고, 함수에서 원본 데이터를 직접 다루며, 이후 학습할 동적 메모리와 자료구조를 이해하는 기반이 된다.

2. 포인터의 기초

2.1. 변수와 메모리 주소

변수는 값을 저장하는 메모리 공간에 붙인 이름이다. 각 변수는 메모리 안에서 고유한 주소를 가진다.

```
1 int number = 10;
2
3 cout << number << endl;
4 cout << &number << endl;
```

number는 저장된 값 10을 의미하고, &number는 해당 변수가 저장된 메모리 주소를 의미한다. 실제 주소는 실행 환경마다 달라진다.

2.2. 포인터 선언

포인터는 메모리 주소를 저장하는 변수이다.

```
1 int number = 10;
2 int* p = &number;
```

int* p는 “정수형 데이터를 가리키는 포인터 p”를 선언한다. p에는 number의 주소가 저장된다.

표현	의미	결과의 종류
number	변수에 저장된 값	int
&number	변수의 주소	int*
p	포인터에 저장된 주소	int*
*p	포인터가 가리키는 위치의 값	int

2.3. 역참조

포인터 앞에 *를 붙이면 포인터가 가리키는 메모리 위치의 값을 읽거나 수정할 수 있다. 이를 역참조(dereference)라고 한다.

```

1 int number = 10;
2 int* p = &number;
3
4 cout << *p << endl;
5
6 *p = 50;
7 cout << number << endl;

```

출력은 다음과 같다.

```

10
50

```

*p = 50;은 포인터 변수 p 자체를 바꾸는 코드가 아니다. p가 가리키는 원본 변수 number의 값을 변경한다.

핵심 포인트

선언에서의 *와 식에서의 *는 모양은 같지만 역할이 다르다.

- int* p;의 *: 포인터 변수 선언
- *p의 *: 가리키는 값에 접근하는 역참조

2.4. 최종 예제: 01_pointer_basic.cpp

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int number = 10;
8     int* p = &number;
9
10    cout << "Value of number : " << number << endl;
11    cout << "Address       : " << &number << endl;
12    cout << "Pointer value  : " << p << endl;
13    cout << "Dereferenced   : " << *p << endl;
14
15    *p = 50;
16
17    cout << "Modified number : " << number << endl;
18
19    return 0;
20 }

```

실습

practice/p01_pointer_basic.cpp

1. 정수형 변수 하나를 선언하고 초기값을 저장한다.
2. 해당 변수의 주소를 저장하는 포인터를 선언한다.
3. 변수의 값, 주소, 포인터에 저장된 주소, 역참조 결과를 출력한다.
4. 포인터를 통해 원본 변수의 값을 수정한다.

3. 배열의 기초

3.1. 배열의 필요성

같은 자료형의 값을 여러 개 저장할 때 변수를 하나씩 선언하면 관리하기 어렵다.

```
1 int score1 = 90;
2 int score2 = 85;
3 int score3 = 70;
```

배열을 사용하면 같은 자료형의 여러 값을 하나의 이름으로 묶을 수 있다.

```
1 int scores[5] = {90, 85, 70, 95, 80};
```

3.2. 인덱스

배열의 각 원소는 인덱스로 접근한다. C++ 배열의 인덱스는 0부터 시작한다.

인덱스	0	1	2	3	4
값	90	85	70	95	80

```
1 cout << scores[0] << endl;
2 cout << scores[3] << endl;
```

위 코드는 각각 90과 95를 출력한다.

3.3. 배열 원소 수정

배열 원소는 일반 변수처럼 대입 연산자로 수정할 수 있다.

```
1 scores[1] = 100;
2 scores[3] = 60;
```

3.4. 반복문을 이용한 배열 순회

배열 원소를 하나씩 직접 출력하는 대신 반복문을 사용할 수 있다.

```
1 for (int i = 0; i < 5; i++)
2 {
3     cout << scores[i] << endl;
4 }
```

조건을 $i \leq 4$ 로 작성해도 같은 범위를 순회하지만, 배열 크기를 직접 드러내는 $i < 5$ 가 일반적으로 더 읽기 쉽다.

핵심 포인트

길이가 5인 배열의 유효한 인덱스는 0부터 4까지이다. `scores[5]`에 접근하면 배열 범위를 벗어나며, 그 결과는 정의되지 않는다.

3.5. 최종 예제: 02_array_basic.cpp

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
```

```

6 {
7     int scores[5] = {90, 85, 70, 95, 80};
8
9     cout << "=== Original Array ===" << endl;
10
11     for (int i = 0; i < 5; i++)
12     {
13         cout << scores[i] << endl;
14     }
15
16     cout << endl;
17
18     scores[1] = 100;
19     scores[3] = 60;
20
21     cout << "=== Modified Array ===" << endl;
22
23     for (int i = 0; i < 5; i++)
24     {
25         cout << scores[i] << endl;
26     }
27
28     return 0;
29 }

```

실습

practice/p02.array_basic.cpp

1. {1, 2, 3, 4, 5}를 저장하는 배열을 만든다.
2. 세 번째 원소를 30으로 수정한다.
3. 다섯 번째 원소를 50으로 수정한다.
4. 반복문으로 배열 전체를 출력한다.

4. 포인터 연산과 배열

4.1. 자료형의 크기: sizeof

sizeof 연산자는 자료형 또는 변수의 메모리 크기를 바이트 단위로 반환한다.

```

1 cout << sizeof(char) << endl;
2 cout << sizeof(int) << endl;
3 cout << sizeof(double) << endl;

```

일반적인 환경에서는 다음과 같이 출력된다.

```

1
4
8

```

다만 C++ 표준은 모든 환경에서 int가 반드시 4바이트라고 보장하지는 않는다. 정확한 크기가 필요하면 실행 환경에서 sizeof로 확인해야 한다.

4.2. 배열의 연속 저장

배열 원소는 메모리에 연속적으로 저장된다. `int`가 4바이트인 환경에서 배열의 첫 주소가 1000이라면 다음 원소들의 주소는 1004, 1008, 1012처럼 이어질 수 있다.

Address	1000	1004	1008	1012	1016
Value	10	20	30	40	50
Index	0	1	2	3	4

4.3. 배열 이름과 첫 번째 원소의 주소

대부분의 식에서 배열 이름은 첫 번째 원소를 가리키는 포인터로 변환된다.

```
1 int numbers[5] = {10, 20, 30, 40, 50};
2
3 cout << numbers << endl;
4 cout << &numbers[0] << endl;
```

두 출력은 같은 주소를 나타낸다. 따라서 다음 두 선언은 같은 위치를 가리킨다.

```
1 int* p1 = numbers;
2 int* p2 = &numbers[0];
```

다만 배열과 포인터가 완전히 같은 자료형이라는 뜻은 아니다. 배열 이름이 많은 식에서 첫 번째 원소의 주소로 변환된다는 의미이다.

4.4. 포인터의 이동

```
1 int numbers[5] = {10, 20, 30, 40, 50};
2 int* p = numbers;
3
4 cout << *p << endl;
5 p++;
6 cout << *p << endl;
```

출력은 10과 20이다. `p++`는 주소를 1바이트 증가시키는 것이 아니라, `int` 원소 한 칸만큼 이동시킨다.

포인터 자료형	<code>p++</code> 의 의미	일반적인 이동량
<code>char*</code>	문자 한 칸 이동	1바이트
<code>int*</code>	정수 한 칸 이동	4바이트
<code>double*</code>	실수 한 칸 이동	8바이트

즉, 포인터 연산은 바이트 수가 아니라 원소 수를 기준으로 해석한다.

핵심 포인트

`p + n`은 주소에 숫자 `n`을 그대로 더하는 것이 아니다. 포인터가 가리키는 자료형의 크기를 고려하여 `n`개의 원소만큼 이동한다.

4.5. 인덱싱과 포인터 표현

배열 인덱싱은 포인터 연산으로 표현할 수 있다.

```
1 numbers[i]
2 *(numbers + i)
```

두 표현은 같은 원소를 의미한다. 포인터 변수 `p`가 배열의 첫 원소를 가리킨다면 다음 역시 같다.

```

1 p[i]
2 *(p + i)

```

예를 들어 다음 코드는 배열 전체를 포인터 표현으로 출력한다.

```

1 for (int i = 0; i < 5; i++)
2 {
3     cout << *(p + i) << " ";
4 }

```

4.6. 포인터 위치를 다시 초기화하기

포인터 자체를 이동시켰다면 현재 위치가 배열의 시작이 아닐 수 있다.

```

1 p += 2;
2 p -= 1;

```

이후 배열 전체를 처음부터 순회하려면 계산을 거꾸로 수행하기보다 다음처럼 명시적으로 다시 대입하는 편이 읽기 쉽다.

```

1 p = numbers;

```

또한 이동과 역참조를 한 줄에 함께 쓰는 코드도 가능하다.

```

1 cout << *(p += 2) << endl;

```

그러나 다음처럼 이동과 출력을 분리하면 실행 흐름을 더 쉽게 읽고 디버깅할 수 있다.

```

1 p += 2;
2 cout << *p << endl;

```

4.7. 최종 예제: 03_pointer_arithmetic.cpp

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout << "=== Size of Data Types ===" << endl;
8     cout << "char   : " << sizeof(char) << " bytes" << endl;
9     cout << "int    : " << sizeof(int) << " bytes" << endl;
10    cout << "double : " << sizeof(double) << " bytes" << endl;
11
12    cout << endl;
13
14    char ch = 'A';
15    int num = 10;
16    double pi = 3.14;
17
18    cout << "=== Size of Variables ===" << endl;
19    cout << "ch   : " << sizeof(ch) << " bytes" << endl;
20    cout << "num  : " << sizeof(num) << " bytes" << endl;
21    cout << "pi   : " << sizeof(pi) << " bytes" << endl;
22
23    cout << endl;
24
25    int numbers[5] = {10, 20, 30, 40, 50};
26

```



```

27     cout << "=== Array Address ===" << endl;
28     cout << "numbers      : " << numbers << endl;
29     cout << "&numbers[0] : " << &numbers[0] << endl;
30
31     cout << endl;
32
33     int* p = numbers;
34
35     cout << "=== Pointer Movement ===" << endl;
36     cout << *p << endl;
37
38     p++;
39     cout << *p << endl;
40
41     p += 2;
42     cout << *p << endl;
43
44     p--;
45     cout << *p << endl;
46
47     cout << endl;
48
49     p = numbers;
50
51     cout << "=== Array Traversal ===" << endl;
52
53     for (int i = 0; i < 5; i++)
54     {
55         cout << *(p + i) << " ";
56     }
57
58     cout << endl;
59
60     return 0;
61 }

```

실습

practice/p03.pointer_arithmetic.cpp

1. {5, 10, 15, 20, 25, 30}을 저장하는 배열을 선언한다.
2. 포인터 이동만 이용하여 5, 15, 10을 차례로 출력한다.
3. 포인터를 배열 시작 위치로 다시 설정한다.
4. *(p + i)를 사용하여 배열 전체를 출력한다.

5. 배열을 함수에 전달하기

5.1. 배열 출력 함수

배열을 여러 번 출력해야 한다면 반복문을 함수로 분리할 수 있다.

```

1 void printArray(int arr[], int size)
2 {
3     for (int i = 0; i < size; i++)
4     {
5         cout << arr[i] << " ";

```

```

6     }
7
8     cout << endl;
9 }

```

호출할 때는 배열 이름과 배열 크기를 함께 전달한다.

```

1 int numbers[5] = {10, 20, 30, 40, 50};
2 printArray(numbers, 5);

```

5.2. 함수 매개변수의 배열 표기

함수 매개변수에서 다음 두 선언은 사실상 같은 의미이다.

```

1 void printArray(int arr[], int size);
2 void printArray(int* arr, int size);

```

배열 전체가 복사되는 것이 아니라 첫 번째 원소의 주소가 함수에 전달된다. 따라서 함수 안의 `arr` 는 원본 배열의 시작 위치를 가리킨다.

5.3. 배열 크기를 함께 전달하는 이유

함수 안으로 전달된 `arr` 는 포인터로 처리되므로 원래 배열의 원소 개수를 자동으로 알 수 없다. 따라서 일반적인 형태에서는 크기를 별도의 매개변수로 함께 전달한다.

```

1 void printArray(int arr[], int size)

```

함수는 `size` 를 이용하여 유효한 범위까지만 반복한다.

5.4. 함수에서 원본 배열 수정

함수는 원본 배열과 같은 메모리를 가리키므로 함수 내부에서 원소를 수정하면 호출한 쪽의 배열도 바뀐다.

```

1 void changeFirst(int arr[])
2 {
3     arr[0] = 999;
4 }

```

```

1 int numbers[5] = {10, 20, 30, 40, 50};
2
3 changeFirst(numbers);
4 printArray(numbers, 5);

```

출력은 다음과 같다.

```
999 20 30 40 50
```

핵심 포인트

배열을 함수에 전달하면 배열 전체가 복사되지 않고 첫 번째 원소의 주소가 전달된다. 따라서 함수는 같은 메모리 공간을 읽고 수정할 수 있다.

5.5. 최종 예제: 04_function_array.cpp

```

1  #include <iostream>
2
3  using namespace std;
4
5  void printArray(int arr[], int size)
6  {
7      for (int i = 0; i < size; i++)
8      {
9          cout << arr[i] << " ";
10     }
11
12     cout << endl;
13 }
14
15 void changeFirst(int arr[])
16 {
17     arr[0] = 999;
18 }
19
20 int main()
21 {
22     int numbers[5] = {10, 20, 30, 40, 50};
23
24     cout << "=== Original Array ===" << endl;
25     printArray(numbers, 5);
26
27     changeFirst(numbers);
28
29     cout << endl;
30     cout << "=== After changeFirst() ===" << endl;
31     printArray(numbers, 5);
32
33     return 0;
34 }

```

실습

practice/p04_function_array.cpp

1. changeLast() 함수를 작성하여 배열의 마지막 원소를 0으로 변경한다.
2. {90, 80, 70, 60, 50, 40} 배열에 함수를 적용한다.
3. addTen() 함수를 작성하여 모든 원소에 10을 더한다.
4. 포인터 표현 $*(p + i)$ 로 변경 전후의 배열을 출력한다.

6. const 포인터

6.1. 왜 const를 사용하는가

포인터를 사용할 때 두 대상을 구분해야 한다.

- 포인터가 가리키는 값
- 포인터 자체에 저장된 주소

const의 위치에 따라 둘 중 하나 또는 둘 다 변경하지 못하게 제한할 수 있다.

6.2. 가리키는 값을 변경할 수 없는 포인터

```
1 const int* p = &number;
```

이 선언에서는 포인터를 통해 값을 변경할 수 없다.

```
1 *p = 20;
```

위 코드는 컴파일 오류가 발생한다. 반면 포인터가 다른 정수 변수를 가리키도록 변경하는 것은 가능하다.

```
1 p = &other;
```

정리하면 값은 잠겨 있고 포인터는 이동할 수 있다.

6.3. 포인터 자체를 변경할 수 없는 포인터

```
1 int* const p = &number;
```

이 선언에서는 포인터에 저장된 주소를 바꿀 수 없다. 선언과 동시에 반드시 초기화해야 한다.

```
1 *p = 20;
```

가리키는 값은 변경할 수 있다. 그러나 다음 코드는 허용되지 않는다.

```
1 p = &other;
```

정리하면 값은 변경할 수 있고 포인터는 잠겨 있다.

6.4. 값과 포인터 모두 변경할 수 없는 경우

```
1 const int* const p = &number;
```

이 선언에서는 역참조를 통한 값 변경도, 포인터의 주소 변경도 허용되지 않는다.

6.5. 세 선언 비교

선언	값 변경	포인터 변경
<code>const int* p</code>	불가능	가능
<code>int* const p</code>	가능	불가능
<code>const int* const p</code>	불가능	불가능

핵심 포인트

별표 *를 기준으로 읽으면 구분하기 쉽다.

- `const int* p`: 별표 왼쪽의 값이 상수
- `int* const p`: 별표 오른쪽의 포인터가 상수
- `const int* const p`: 양쪽 모두 상수

6.6. 최종 예제: 05_const_pointer.cpp

```

1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int num = 10;
8     int other = 20;
9
10    const int* p1 = &num;
11    cout << "*p1 = " << *p1 << endl;
12
13    p1 = &other;
14    cout << "*p1 = " << *p1 << endl;
15
16    cout << endl;
17
18    int* const p2 = &num;
19    *p2 = 100;
20    cout << "num = " << num << endl;
21
22    cout << endl;
23
24    const int* const p3 = &num;
25    cout << "*p3 = " << *p3 << endl;
26
27    return 0;
28 }

```

실습

practice/p05_const_pointer.cpp

1. const int* 포인터가 다른 변수의 주소를 가리키게 한 뒤 값을 출력한다.
2. int* const 포인터로 원본 값을 수정한다.
3. 세 가지 const 포인터의 값 변경 가능 여부와 포인터 변경 가능 여부를 출력한다.

7. 프로젝트 파일 구성

```

cpp/
├── day03/
│   ├── 01_pointer_basic.cpp
│   ├── 02_array_basic.cpp
│   ├── 03_pointer_arithmetic.cpp
│   ├── 04_function_array.cpp
│   ├── 05_const_pointer.cpp
│   └── practice/
│       ├── p01_pointer_basic.cpp
│       ├── p02_array_basic.cpp
│       ├── p03_pointer_arithmetic.cpp
│       ├── p04_function_array.cpp
│       └── p05_const_pointer.cpp
└── cpp_day03.tex

```

```
├── cpp_day03.pdf
└── README.md
```

개념별 예제 파일은 Day 3 루트에 번호를 붙여 분리하고, 실습 파일은 `practice` 폴더에 같은 번호 체계로 저장한다.

8. 핵심 정리

1. 포인터는 메모리 주소를 저장하는 변수이다.
2. `&variable`은 변수의 주소를 얻고, `*pointer`는 포인터가 가리키는 값에 접근한다.
3. 포인터를 역참조하여 값을 대입하면 원본 변수가 변경된다.
4. 배열은 같은 자료형의 여러 값을 연속된 메모리 공간에 저장한다.
5. 배열 이름은 대부분의 식에서 첫 번째 원소를 가리키는 포인터로 변환된다.
6. 포인터 연산은 바이트가 아니라 가리키는 자료형의 원소 단위로 이동한다.
7. `arr[i]`와 `*(arr + i)`는 같은 원소를 의미한다.
8. 배열을 함수에 전달하면 첫 번째 원소의 주소가 전달된다.
9. 함수에서 배열 원소를 수정하면 원본 배열도 변경된다.
10. 함수에 전달된 배열의 크기는 자동으로 보존되지 않으므로 일반적으로 크기를 함께 전달한다.
11. `const int*`는 값을, `int* const`는 포인터를 변경하지 못하게 한다.
12. `const int* const`는 값과 포인터 모두 변경하지 못하게 한다.

체크 질문

다음 질문에 답할 수 있다면 Day 3의 핵심을 이해한 것이다.

1. `p`와 `*p`의 차이는 무엇인가?
2. `int* p`에서 `p++`는 주소를 몇 바이트 이동시키는가?
3. 왜 `numbers[3]`과 `*(numbers + 3)`이 같은가?
4. 배열을 함수에 전달했을 때 원본 배열이 수정되는 이유는 무엇인가?
5. `const int*`와 `int* const`는 무엇이 다른가?